Experiment No. 10

CODING FOR THE PROJECT AFTER TESTING

Doctor Appointment Booking System — Aurelia Care

Student details	Information
Name	Ronit Rajput
Roll No.	E052
Program	Computer Science and Business Systems
Division / Batch	E / 2
Date of Experiment	26 October 2026
Date of Submission	_____
Grade	_____

PART A

A.1 Aim

Coding for the project after testing: identify a verified error, correct the implementation, repeat the affected tests, and deploy the corrected application in the laboratory environment.

A.2 Prerequisites

Requirement engineering, data-flow diagrams (DFD), control-flow diagrams (CFD), and test cases.

A.3 Outcome

Corrected project code, regression tests, a running local deployment, and documented verification evidence.

A.4 Theory

Post-testing development is a controlled feedback cycle: reproduce an observed failure, identify its cause, make a focused correction, add regression protection, rebuild, and verify the deployed behaviour. Expected validation errors must be distinguished from genuine application defects.

A DFD supports understanding of data movement between the user, API, authentication service, and database. A CFD identifies decisions such as origin validation, credential verification, and session creation. Test cases connect those decisions with expected and observed outcomes.

Task to be completed

- Highlight the errors identified after testing.
- Make changes in the coding.
- Deploy the project.

PART B

B.1 Tasks Given in Part A

1. Project and implementation

Aurelia Care provides role-based login, doctor search, appointment scheduling, simulated payments, and Excel exchange. React builds the interface; FastAPI handles the API; SQLite stores users, sessions, slots, appointments, and payments.

Component	Implementation / responsibility
Frontend	frontend/src/: login, doctor profiles, booking dialog, dashboards
API and origin safeguard	backend/app/main.py: request validation, routes, trusted browser origins
Authentication	backend/app/security.py: scrypt passwords, JWT cookie, revocable sessions
Booking	backend/app/appointments.py: availability checks and transactional writes
Persistence	backend/app/database.py: SQLite connection and transactions
Launcher	start.sh: dependencies, frontend build, database initialization, server startup
New regression test	tests/test_origin_configuration.py: five independent configuration cases

2. Error identified after testing — DEF-10-01

Field	Recorded observation
Title	Login blocked when the application uses a custom local port
Requirement	The launcher supports PORT; browser login on that configured local port should work by default.
Preconditions	PORT=8010; no explicit ALLOWED_ORIGINS; active fictional patient account
Reproduction	Start server on port 8010; open its login page; submit valid credentials.
Expected	HTTP 200, authenticated patient session, and patient dashboard
Actual before correction	HTTP 403: “This origin is not allowed.” No successful login.
Severity / impact	High functional impact for custom-port users; default port 8000 remained usable.
Root cause	The default trusted-origin list hard-coded port 8000 while the launcher used PORT.
Evidence	One failing custom-port regression case and a fresh browser screenshot of the rejection.

This is one confirmed application defect. Wrong passwords, blocked untrusted origins, and deliberately failed simulated payments are expected behaviours, not additional defects. The earlier Experiment 9 alternate-port observation prompted this reproduction.

3. Coding Changes

3.1 Before correction

```
origins = [
    x.strip()
    for x in os.getenv(
        "ALLOWED_ORIGINS",
        "http://localhost:5173,http://127.0.0.1:5173,"
        "http://localhost:8000,http://127.0.0.1:8000",
    ).split(",")
]
```

The excerpt is line-wrapped for this report. Both the default application origins were fixed at port 8000.

3.2 After correction

```
app_port = os.getenv("PORT", "8000")
origins = [
    x.strip()
    for x in os.getenv(
        "ALLOWED_ORIGINS",
        "http://localhost:5173,http://127.0.0.1:5173,"
        f"http://localhost:{app_port},http://127.0.0.1:{app_port}",
    ).split(",")
]
```

3.3 Explanation of the correction

- Read the application port from PORT, with 8000 as the default.
- Construct the two local application origins using that port. The existing Vite development origins remain available by default.
- Preserve an explicitly supplied ALLOWED_ORIGINS value, which takes precedence over the generated defaults.
- Retain the rejection of unrelated browser origins. The fix does not introduce a wildcard.
- Document the alternate-port Mac launcher command and add five configuration regression cases.

Version	Source reference
Before-fix baseline	335d23cae4ce143f260fd3eea77e7e40db66a2c8
Code correction commit	9dae96f92a0eac31b56128bb19b35a9ef2dfaa7c
Changed tracked files	backend/app/main.py; tests/test_origin_configuration.py; README.md

Each new configuration test launches an isolated Python process so environment-variable settings are applied before the application is imported. Tests use temporary databases and fictional credentials; existing user data is preserved.

4. Retesting and Regression Verification

4.1 Configuration cases before and after the code change

Case	Configuration / expected result	Before fix	After fix
CFG-01	No PORT; localhost:8000 login → 200	Pass	Pass
CFG-02	PORT=8010; 127.0.0.1:8010 login → 200	FAIL: 403	Pass: 200
CFG-03	PORT=8010; untrusted origin → 403	Pass	Pass
CFG-04	Explicit site override; local origin → 403	Pass	Pass
CFG-05	Explicit site override; specified site → 200	Pass	Pass

Successful cases also assert a patient response and the exact Access-Control-Allow-Origin header. The rejected cases assert HTTP 403. No checks were removed or weakened to make the suite pass.

4.2 Completed validation

Check	Observed result
New tests before correction	4 passed; 1 failed — 4.49 seconds
Full backend suite after correction	26 passed; 0 failed; 0 skipped — 17.72 seconds
Regression scope	21 existing workflow tests plus 5 new origin-configuration tests
Frontend production build	TypeScript and Vite build completed successfully
Actual Bash launcher	Dependency installation, frozen npm install, build, initialization, and server startup completed
Shell syntax	start.sh, Start-Mac.command, and scripts/cloud-setup.sh passed bash -n
Fresh browser scenarios	4 passed: wrong-password rejection, valid login, failed simulated payment, successful retry / confirmation
Deployment health	HTTP 200; status ok; database 1; payment mode simulation
Frontend serving	HTTP 200 with the compiled React root present

4.3 Environment troubleshooting

The initial cloud launcher attempt could not create npm’s default cache under the restricted home directory. The launcher was rerun with npm_config_cache=/workspace/npm-cache, a writable cache. This is a cloud-environment adjustment, separate from DEF-10-01; it did not require changing application logic.

An initial test process started before local networking permission was available was stopped. The completed before/after runs shown above were executed after the prerequisite was supplied. Pytest reported an upstream Starlette/AnyIO deprecation warning; it did not fail the tests.

5. Before-and-After Browser Evidence

These screenshots were captured from the real application on port 8010 with a separate fictional laboratory database. Generated installation passwords are omitted.

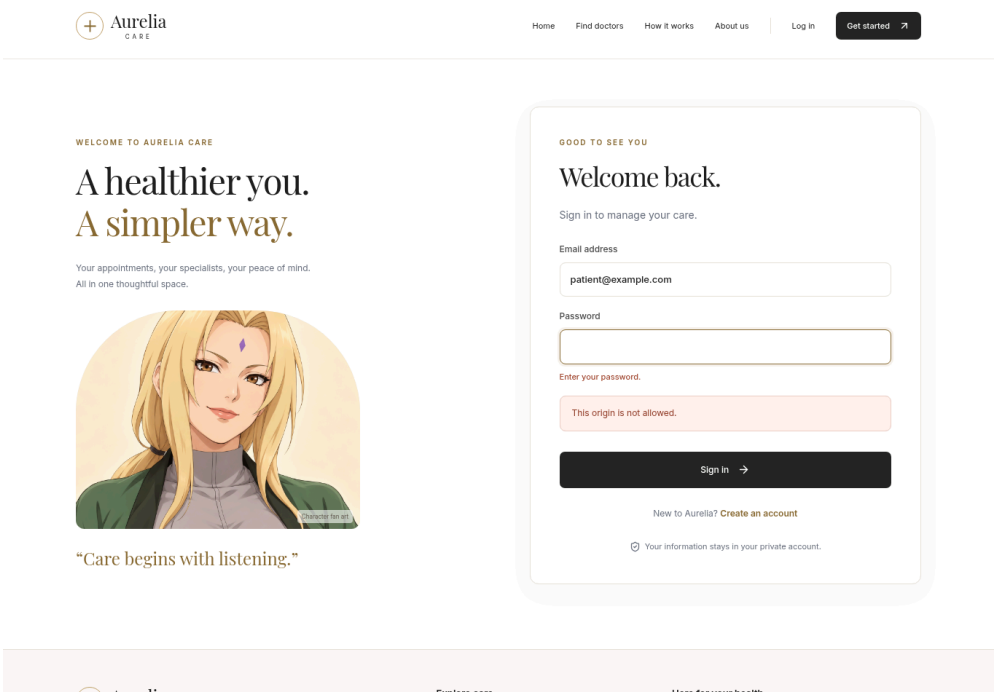


Figure 1. Before correction: valid credentials were blocked by the origin safeguard (HTTP 403).

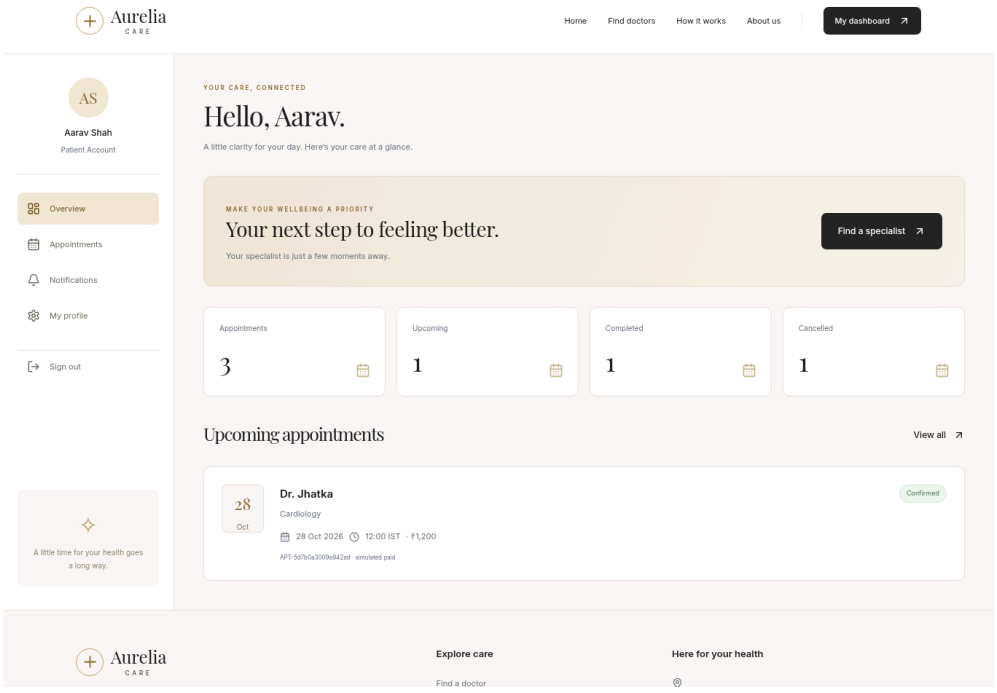


Figure 2. After correction: the same lab installation accepts login and opens the patient dashboard.

6. Deployment of the Corrected Project

The corrected application was deployed and exercised in the Linux cloud laboratory. This is a local demonstration deployment; no public website URL or production hosting is claimed. GitHub publication distributes the source and reports, not a running backend.

6.1 Executed deployment command

```
PORT=8010 \
DB_PATH=/workspace/experiment10/browser/database/lab.sqlite3 \
npm_config_cache=/workspace/npm-cache \
bash start.sh
```

ALLOWED_ORIGINS was intentionally not set. The successful same-origin browser login therefore verifies that the PORT-based default correction is active. The existing launcher bound the application to loopback and served the production frontend and API from one process.

6.2 Health check observed after startup

```
{
  "status": "ok",
  "database": 1,
  "timezone": "Asia/Kolkata",
  "local_time": "2026-10-26T07:55:19+05:30",
  "payments": "simulation"
}
```

The homepage also returned HTTP 200 with the compiled React root. The browser then completed login, a failed payment attempt, and a successful booking retry.

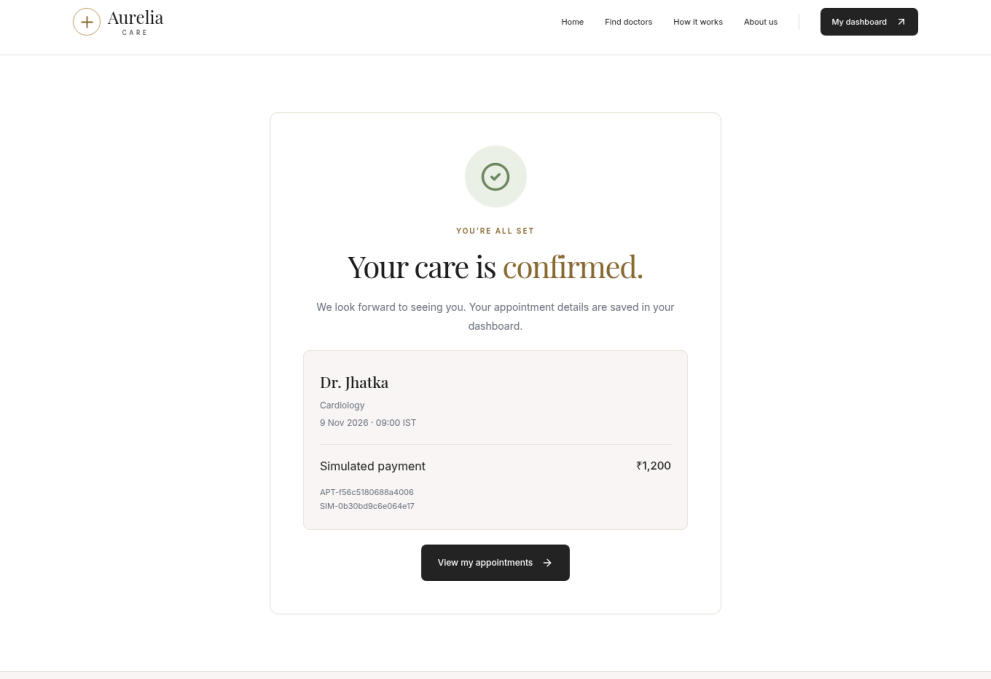


Figure 3. A confirmed booking after the corrected application was started through its Bash launcher.

7. Running the Updated Project on a Mac

Download the source ZIP for the codex/experiment10-report branch, extract it, open Terminal in the extracted folder, and run:

```
PORT=8010 bash start.sh
```

After startup, open http://localhost:8010 in the Mac browser and keep Terminal running. Locally generated credentials are stored in DEMO_ACCOUNTS.md. Python 3.12+ and Node.js 22+ are required. The cloud npm-cache override above is specific to this lab and is not normally needed on a Mac.

To preserve an existing installation, stop it, keep a backup, and copy its entire database folder and DEMO_ACCOUNTS.md into the updated folder before starting. Do not copy an actively running SQLite database.

7.1 Future public deployment configuration

A public deployment requires a Python-capable host with persistent storage, HTTPS, APP_ENV=production, SEED_DEMO=0, a strong secret, an explicit trusted HTTPS origin, and a persistent DB_PATH. These are deployment requirements, not completed production tasks. Full instructions remain in docs/DEPLOYMENT.md.

B.2 Observations and Learning

- Passing default-port tests did not establish that a configurable port worked. Configuration variants need their own regression cases.
- The 403 response came from the origin safeguard before password verification; the account credentials were not the cause.
- Changing the generated local origin list corrected the defect while explicit overrides and untrusted-origin rejection remained effective.
- Subprocess tests isolated import-time configuration so each environment combination was tested independently.
- The actual launcher and real browser checks connected unit/API evidence to the deployed application.
- A writable npm cache solved an environment restriction without changing dependencies or bypassing integrity checks.

B.3 Conclusion

The aim was achieved by reproducing a custom-port login defect, correcting the implementation, adding five regression tests, and deploying the corrected application in the local cloud laboratory. The full 26-test backend suite passed; the frontend built successfully; four browser scenarios passed; and the deployed application returned healthy API and frontend responses.

Results apply to the tested Linux lab configuration. Native Mac execution, public hosting, load capacity, and production uptime were not verified in this experiment.

B.4 Question of Curiosity

1. Which different tests were performed for your project before deploying the software?

The project was checked at several levels. Existing workflow tests were rerun with five new configuration tests, followed by production-build, launcher, health, and browser smoke checks. Deployment smoke tests also verified the running artifact after startup.

Test category	Project checks and evidence
Functional / black-box API tests	Login, registration, validation errors, doctor filtering, booking, cancellation, rescheduling, completion, and notifications.
Configuration regression tests	Default port, alternate port, explicit allowed-origin override, and rejection of an unrelated origin; 5 cases.
Authentication / authorization tests	Password hashing, cookie/session handling, logout revocation, role restrictions, and origin enforcement.
Database integration tests	Appointment persistence, slot state, simulated payment records, and Excel import/export integrity.
Negative / error-handling tests	Wrong credentials, invalid fields, unavailable and past slots, failed simulated payments, and invalid workbook data.
Concurrency test	Eight requests compete for one slot: one success and seven conflicts.
White-box-informed checks	Tests exercise login rejection/success, origin guard outcomes, booking decisions, and atomic rescheduling conflicts. No new source-coverage percentage is claimed for Experiment 10.
Frontend build validation	TypeScript checking and Vite production compilation completed.
Browser smoke tests	Wrong-password error, successful login/dashboard, failed payment, and successful retry/booking confirmation.
Deployment checks	Real launcher execution, shell syntax checks, health endpoint, compiled frontend response, and browser interaction on port 8010.

The current backend total is 26 tests, not the sum of every assertion or category in the table. Several tests cover multiple categories. The earlier Experiment 9 focused suite is separate historical evidence and is not added to this run’s count.

Evidence and references

- SE Lab Manual, Experiment 10: aim, prerequisites, tasks, Part B sections, and curiosity question.
- Source baseline 335d23c; correction commit 9dae96f; code-change.diff records the application correction.
- before-fix.xml / .txt: one failure reproduced; after-fix.xml / .txt: all 26 backend tests passed.
- browser-before.json, browser-results.json, health.json, screenshots, and build/deployment logs document actual observations.